

PROGRAMAÇÃO IV:

Aula 02



Prof. Roberson Alves

OBJETIVOS DA AULA

- HTML Semântico
- Introdução ao Javascript
- Javascript e Typescript
- Introdução ao Typescript

O que é HTML Semântico?

- Uso de tags que descrevem o conteúdo
- Aumenta a acessibilidade, manutenibilidade e SEO
- Substitui divs genéricas por elementos com significado

Por que usar HTML Semântico?

- Melhora a compreensão do código
- Ajuda leitores de tela e ferramentas assistivas
- Beneficia motores de busca
- Facilita a colaboração entre desenvolvedores

Exemplos de Tags Semânticas

- `<header>` – Cabeçalho da página
- `<nav>` – Navegação
- `<main>` – Conteúdo principal
- `<section>` – Seção independente
- `<article>` – Conteúdo reutilizável
- `<aside>` – Conteúdo secundário
- `<footer>` – Rodapé
- `<figure>`, `<figcaption>` – Imagens e legendas

Antes vs. Depois (com e sem semântica)

-  HTML sem semântica:

```
<div id="topo">
```

```
  <div class="menu">...</div>
```

```
</div>
```

-  HTML semântico:

```
<header>
```

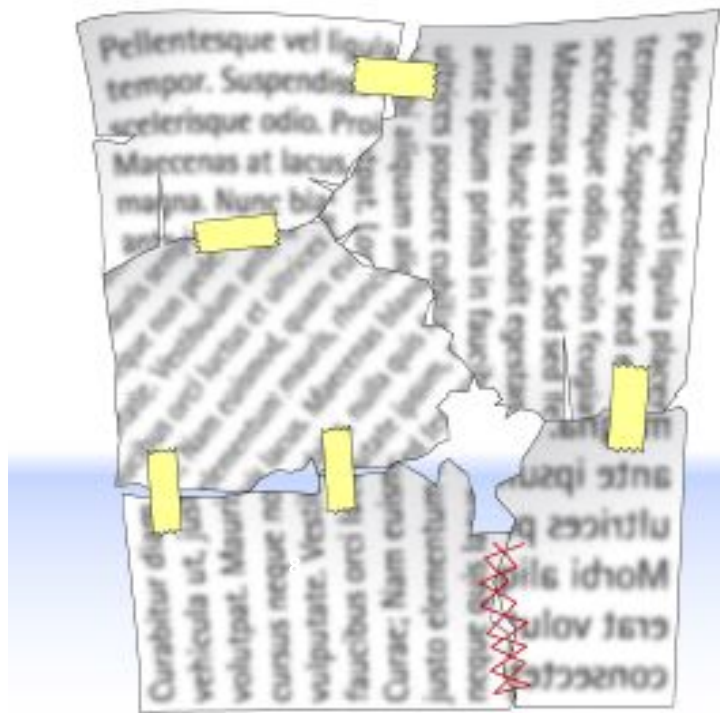
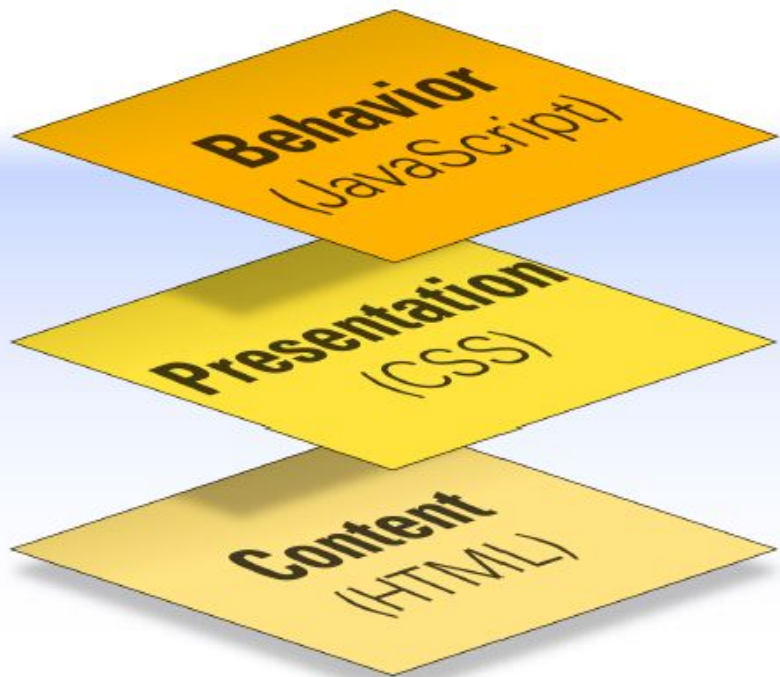
```
  <nav>...</nav>
```

```
</header>
```

Dicas de uso de HTML Semântico

- Use `<section>` para dividir blocos
- Use `<article>` para conteúdo independente
- Evite `<div>` quando há uma tag semântica equivalente
- Combine com boas práticas de CSS e acessibilidade

INTRODUÇÃO AO JAVASCRIPT

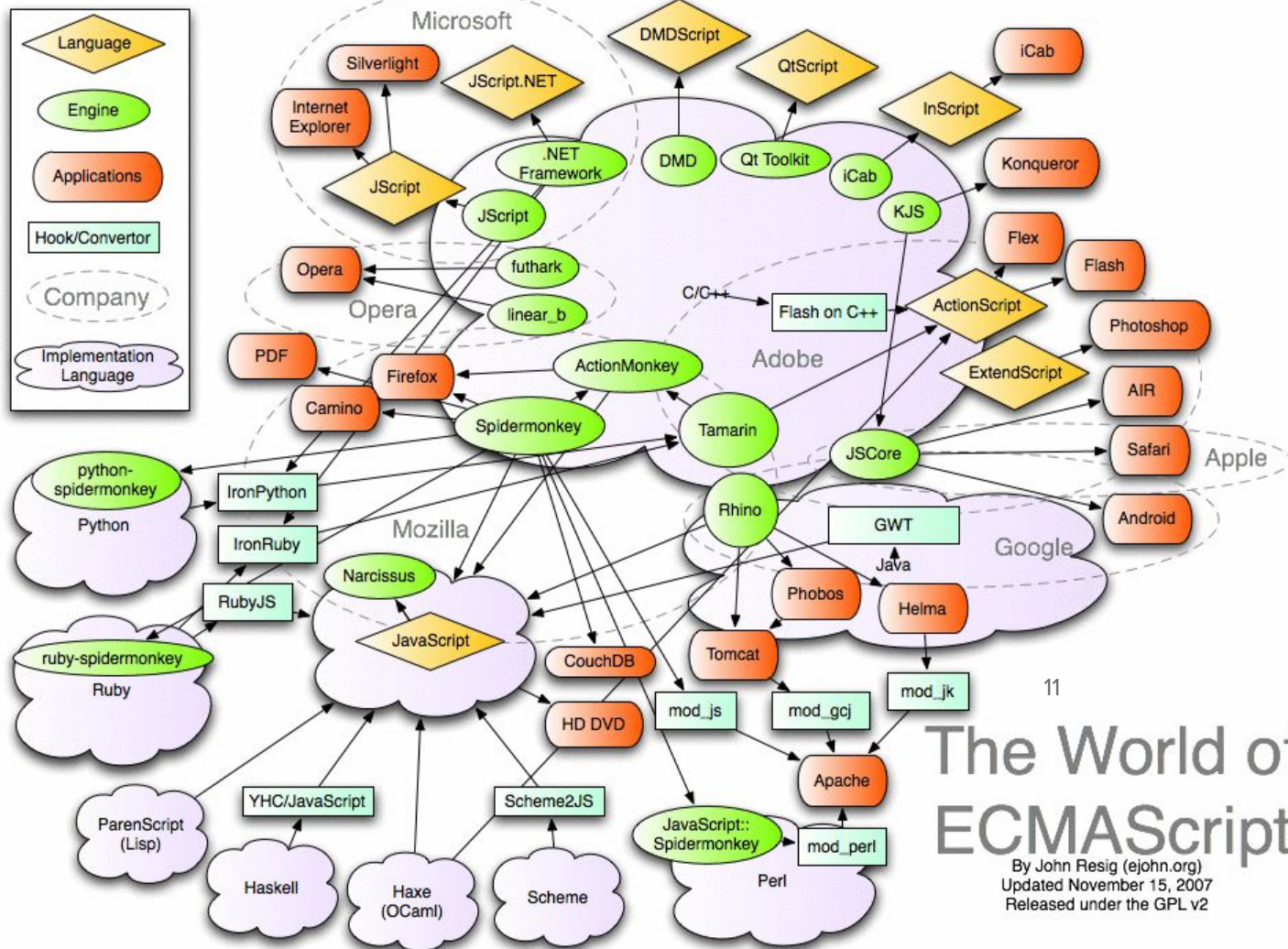


O QUE É JAVASCRIPT?

- Linguagem de Scripting Dinâmica com características OO
- Executada, **principalmente**, no **navegador**
- Linguagem + **popular** do mundo;
- Linguagem + **incompreendida**;
- Serve para tudo:
 - Programas completos;
 - Acesso remoto;
 - Games;
 - Animações;
 - Etc.

HISTÓRIA DO JAVASCRIPT

- Desenvolvida por Brendan Eich na Netscape em 1995
 - Scripting language for Navigator 2
- Padronizada para compatibilidade com navegadores
 - ECMAScript 2021/2022
- Relacionada ao Java apenas pelo nome
 - Name was part of a marketing deal
- Várias implementações disponíveis(+40)
 - Para VMs
 - Embarcáveis



The World of ECMAScript

By John Resig (ejohn.org)
 Updated November 15, 2007
 Released under the GPL v2

CARACTERÍSTICAS

- É uma linguagem poderosa, com sua grande aplicação (não é a única) do lado cliente (browser);
- É uma linguagem de scripts que permite interatividade nas páginas web;
- É incluída na página HTML e interpretada pelo navegador;
- É simples, porém pode-se criar construções complexas e criativas;
- Multiplataforma;
- Padronizada: **ECMAScript**



13

**JavaScript é uma linguagem
imperativa**



14

JavaScript NÃO é JAVA



15

Mais uma vez:
JavaScript NÃO é JAVA



16

Só para deixar claro:
JavaScript NÃO é JAVA

ALGUMAS COISAS QUE SE PODE FAZER COM JS

- Validar entrada de dados em formulários: campos não preenchidos ou preenchidos incorretamente poderão ser verificados;
- Realizar operações matemáticas e computação;
- Abrir janelas do navegador, trocar informações entre janelas, manipular propriedades como histórico, barra de status, plug-ins, applets e outros objetos;
- Interagir com o conteúdo do documento tratando toda a página como uma estrutura de objetos;
- Interagir com o usuário através do tratamento de eventos;
- Rodar aplicações desktop e mobile;
- ...

JAVASCRIPT NÃO OBSTRUTIVO

- Que o conteúdo da página deve estar presente e funcional, ainda que se perca em usabilidade, caso o usuário esteja visualizando o documento em um dispositivo (por exemplo, navegador) sem suporte para JavaScript;
- Usar a linguagem com vistas a unicamente incrementar a usabilidade da página;
- Escrever scripts em arquivos externos para serem linkados ao documento e não inserir script na marcação HTML; e
- JavaScript X Acessibilidade.

ALGUMAS BIBLIOTECAS



FORMAS DE USO

- Dentro próprio código HTML:

```
<a href="#" onclick="alert('alô mundo!')">Diga alô</a>
```

- Separado em uma tag de script (preferencialmente dentro da tag <head></head>):

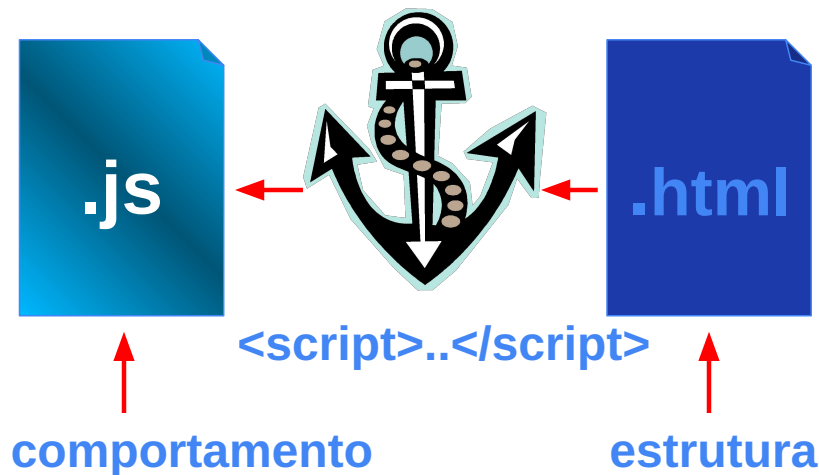
```
<script type="text/javascript">  
    alert("alô mundo");  
</script>
```

- Mais separado ainda dentro de um arquivo “texto” com extensão .js sendo chamado por uma tag script:

```
<script type="text/javascript" src="script.js"></script>
```

DOIS ARQUIVOS SEPARADOS?

- Usaremos dois arquivos texto:
 - Um com HTML com extensão .html
 - Outro com JavaScript com extensão .js
 - Haverá ainda uma tag HTML que “unirá” os arquivos



ALÔ MUNDO - VERSÃO 1

```
<html>
<head>
  <title>Alô!</title>
  <script type="text/javascript">
    alert("Alô mundo!");
  </script>
</head>
<body>
  ...
</body>
</html>
```

ALÔ MUNDO – VERSÃO 2

```
...  
<head>  
  <title>Alo!</title>  
  <script type="text/javascript" src = "alomundo.js">  
  </script>  
</head>  
...
```

 **alomundo.html**

```
alert("alo mundo");
```

 **alomundo.js**

SINTAXE

- Tudo é case-sensitive, ou seja: **teste** é diferente de **Teste**
- Construções simples: após cada instrução, finaliza-se utilizando um ponto-e-vírgula:

Instrução1;

Instrução2;

- Ex:

```
alert("alô");
```

```
alert("mundo");
```


SINTAXE

- Comentários de uma linha:

```
alert("teste"); // comentário de uma linha
```

- Comentário de várias linhas:

```
/* este é um comentário  
de mais de uma linha */
```

- Saída de dados: em lugar de usar a função alert, podemos utilizar:

```
document.write("<h1>teste</h1>");
```

- Onde **document** representa a própria página e write escreve no seu corpo.

VARIÁVEIS

- Variáveis são usadas para armazenar valores temporários;
- Usamos a palavra reservada `var` ou `let` para defini-las;
- Em JS, as variáveis são fracamente tipadas, ou seja, o tipo não é definido explicitamente e sim a partir de uma atribuição (=)
- Ex:

```
let a = 'hello'; // globally scoped
var b = 'world'; // globally scoped
console.log(window.a); // undefined
console.log(window.b); // 'world'
var a = 'hello';
var a = 'world'; // No problem, 'hello' is replaced.
let b = 'hello';
let b = 'world'; // SyntaxError: Identifier 'b' has already been declared
```

ALGUNS TIPOS

- **Números: inteiros e decimais**

```
var i = 3;
```

```
var peso = 65.5;
```

```
var inteiroNegativo = -3;
```

```
var realNegativo = -498.90;
```

```
var expressao = 2 + (4*2 + 20/4) - 3;
```

- **Strings ou cadeia de caracteres:**

```
var nome = "josé";
```

```
var endereco = "rua" + " das flores";
```

```
nome = nome + " maria";
```

```
endereco = "rua a, numero " + 3;
```

ALGUNS TIPOS

- **Arrays:** alternativa para o armazenamento de conjuntos de valores

```
var numeros = [1,3,5];
```

```
var strNumeros = [];
```

```
strNumeros[0] = "First";
```

```
strNumeros[1] = "Second";
```

```
var cidades = [ ];
```

```
cidades[0] = "Parnaíba";
```

```
cidades[1] = "Teresina";
```

```
cidades[2] = "LC";
```

```
alert("A capital do Piauí é " + cidades[1]);
```



ALGUNS TIPOS

- Tamanho de um array: usamos a propriedade `length` do próprio array

```
alert(cidades.length);
```

- Último item de um array:

```
alert(cidades[cidades.length-1]);
```

ALGUNS TIPOS

- Arrays associativos:

- baseados também na ideia `array[indice] = valor`
- O índice/chave de um array associativo é geralmente uma string

```
var idades = [];  
idades["ely"] = 29;  
idades["Gleison"] = 20;  
idades["maria"] = 20;  
alert("Minha idade é: " + idades["maria"]);
```

ALGUNS TIPOS

- Lógico: tipo que pode ter os valores true ou false

```
var aprovado = true;  
alert(aprovado);
```

OPERADOR DE TIPOS

- **typeof:** inspecionar o tipo de uma variável ou valor:

```
var a = "teste";  
alert( typeof a); // string  
alert( typeof 95.8); // number  
alert( typeof 5); // number  
alert( typeof false); // boolean  
alert( typeof true); // boolean  
alert( typeof null); // object  
var b;  
alert( typeof b); // undefined
```

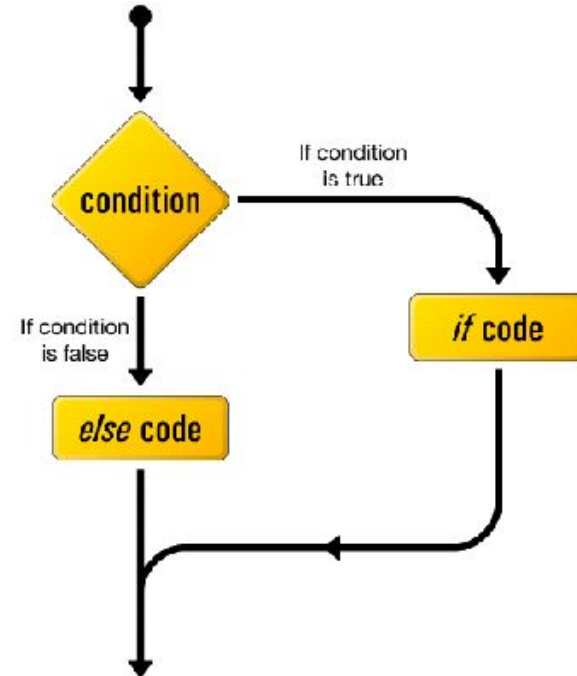

OPERADOR DE TIPOS

- Utilizando **typeof** podemos ter os seguintes resultados:
 - **undefined**: se o tipo for indefinido.
 - **boolean**: se o tipo for lógico
 - **number**: se for um tipo numérico (inteiro ou ponto flutuante)
 - **string**: se for uma string
 - **object**: se for uma referência de tipos (objeto) ou tipo nulo

ESTRUTURAS DE DECISÃO – IF E ELSE

- Sintaxe:

```
if (condição) {  
    código da condição verdadeira;  
} else {  
    código da condição falsa;  
}
```



{ simboliza um início/begin
}

} representa um fim/end

OPERADORES CONDICIONAIS E LÓGICOS

>	A > B
>=	A >= B
<	A < B
<=	A <= B
==	A == B
!=	A != B

← A é igual a B

← A é diferente de B

- && : and
- || : or
- ! : not

```
var idade = 17;  
if (idade >= 16 && idade < 18) {  
    alert("voto facultativo");  
}
```

TABELA DE OPERADORES E PRIORIDADE

Prioridade



Operador	Descrição
. [] ()	Acesso a propriedades, indexação, chamadas a funções e sub-expressões
++ -- ~ ! new delete typeof	Operadores unários e criação de objetos
* / %	Multiplicação, divisão, módulo
+ -	Adição, subtração, concatenação de strings
<< >> >>>	Deslocamento de bit
< <= > >= instanceof	Menor, menor ou igual, maior, maior ou igual, instanceof
== != === !==	Igualdade, desigualdade, igualdade estrita, e desigualdade estrita
&	AND bit a bit
^	XOR bit a bit
	OR bit a bit
&&	AND lógico
	OR lógico
? :	Operador condicional (ternário)

ESTRUTURAS DE DECISÃO – IF E ELSE

```
if (navigator.cookieEnabled) {  
    alert("Seu navegador suporta cookies");  
} else {  
    alert("Seu navegador não suporta cookies");  
}
```

ESTRUTURAS DE DECISÃO – SWITCH

```
switch (expressão) {  
    case valor 1:  
        //código a ser executado se a expressão = valor 1;  
        break;  
    case valor 2:  
        //código a ser executado se a expressão = valor 2;  
        break;  
    ...  
    case valor n:  
        //código a ser executado se a expressão = valor n;  
        break;  
    default:  
        //executado caso a expressão não seja nenhum dos valores;  
}
```

ESTRUTURAS DE DECISÃO – SWITCH

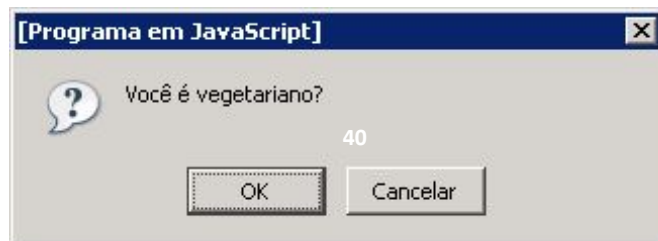
```
var idade = 20;

switch (idade) {
    case (29):
        alert("Você está no auge.");
        break;
    case (40) :
        alert("A vida começa aqui.");
        break;
    case (60) :
        alert("Iniciando a melhor idade.");
        break;
    default:
        alert("A vida merece ser vivida, não importa a idade.");
        break;
}
```

JANELAS DE DIÁLOGO - CONFIRMAÇÃO

- Nos permite exibir uma janela popup com dois botões: ok e cancel
- Funciona como uma função:
 - Se o usuário clicar em ok, ela retorna true; em cancel retorna false
- Ex:

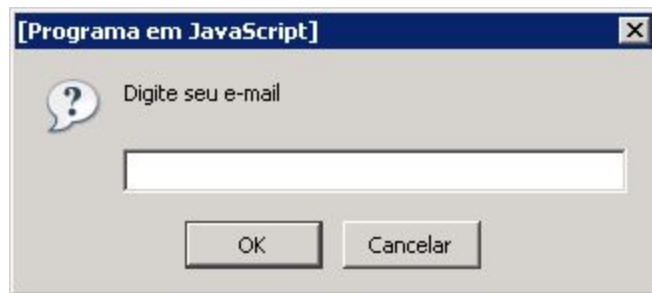
```
var vegetariano = confirm("Você é vegetariano?");  
  
if (vegetariano == true) {  
    alert("Coma mais proteínas");  
}  
  
else {  
    alert("Coma menos gordura");  
}
```



JANELAS DE DIÁLOGO – PROMPT

- Nos permite exibir uma janela pop up com dois botões (ok e cancel) e uma caixa de texto;
- Funciona como uma função: se o usuário clicar em ok e preencher a caixa de texto, ela retorna o valor do texto; em cancel retorna null;
- O segundo parâmetro pode ser preenchido como uma sugestão;
- Ex:

```
var email = prompt("Digite seu e-mail","");  
alert("O email " + email + " será usado para spam.");
```



JANELAS DE DIÁLOGO – PROMPT

- O que lemos da janela prompt é uma string;
- Podemos converter strings para inteiro utilizando as funções pré-definida `parseInt` e `parseFloat`;
- `parseInt(valor, [base])`: converte uma string para inteiro.
 - O valor será convertido para inteiro e base é o número da base (vamos usar base 10)
- `parseFloat(valor, [base])`: converte uma string para um valor real/ponto flutuante

JANELAS DE DIÁLOGO – PROMPT

- Ex:

```
var notaStr = prompt("Qual a sua nota?","");  
var trabStr = prompt("Qual o valor do trabalho?","");  
var nota = parseFloat(notaStr,10);  
var trab = parseFloat(trabStr,10);  
nota = nota + trab;  
alert("Sua nota é: " + nota );
```

ESTRUTURAS DE REPETIÇÃO – FOR

- Executa um trecho de código por uma quantidade específica de vezes
- Sintaxe:

```
for (inicio; condicao; incremento/decremento) {  
    // código a ser executado.  
}
```

- Ex:

```
var numeros = [1, 2, 3, 4, 5];  
for (var i = 0; i < numeros.length; i++) {  
    numeros[i] = numeros[i]* 2;  
    document.write(numeros[i] + "<br/>");  
}
```

AÇUCARES SINTÁTICOS

- Em JS podemos utilizar formas “compactada” instruções:

`numero = numero + 1` equivale a `numero++`

`numero = numero - 1` equivale a `numero--`

`numero = numero + 1` equivale a `numero += 1`

`numero = numero - 1` equivale a `numero -= 1`

`numero = numero * 2` equivale a `numero *= 2`

`numero = numero / 2` equivale a `numero /= 2`

PRÁTICA

- **Elabore scripts usando a função prompt que:**
 - **Leia um valor e imprima os resultados: “É maior que 10” ou “Não é maior que 10” ou ainda “É igual a 10”**
 - **Some dois valores lidos e imprima o resultado;**
 - **Leia 2 valores e a operação a ser realizada (+, -, * ou /) e imprima o resultado (use um switch)**
 - **Leia um nome e um valor n e imprima o nome n vezes usando o laço for**

ESTRUTURAS DE REPETIÇÃO - WHILE

- Executa um trecho de código enquanto uma condição for verdadeira
- Sintaxe:

```
while (condicao) {  
    //código a ser executado  
}
```

Ex:

```
var numero = 1;  
while (numero <= 5) {  
    alert("O número atual é: " + numero);  
    numero = numero + 1;  
}
```

ESTRUTURAS DE REPETIÇÃO – DO...WHILE

- Executa um trecho de código enquanto uma condição for verdadeira
- Mesmo que a condição seja falsa, o código é executado pelo menos uma vez
- Sintaxe:

```
do {  
    // código a ser executado.  
} while (condição) ;
```

Ex:

```
var numero = 1;  
do {  
    alert("O número atual é: " + numero);  
    numero = numero + 1;  
} while (numero <= 5) ;
```


TRATAMENTO DE ERROS - TRY CATCH

- Erros comuns devem ser tratados/evitados;
- Testar para verificar possíveis erros comuns;
- Exceções podem ser tratadas com: try, catch e finally;

```
/* Considerando a função do exemplo anterior já declarada */

try {
    lerarr(); //função que pode lançar exceção
} catch (e) {
    // comandos a serem executados em caso de exceção
    window.alert("Oops! No soup for you!");
} finally {
    document.write("ocorreu um erro");
}
```

- Acessar o tipo/nome e mensagem pelas propriedades: name e message

FUNÇÕES

- Funções são blocos de código reutilizáveis;
- Elas não são executadas até que sejam chamadas;
- Podem ter parâmetros de entrada e de saída;
- Podemos ter vários parâmetros de entrada separados por vírgulas;
- Podemos retornar um valor através da instrução `return`.

FUNÇÕES

- Sintaxe:

```
function nomeDaFuncao() {  
    //códigos referentes à função.
```

```
    ...
```

```
}
```

```
function nomeDaFuncao(p1, p2, p3, ...) {  
    //códigos referentes à função.
```

```
    ...
```

```
}
```

```
function nomeDaFuncao(p1, p2, p3, ...) {
```

```
    ...
```

```
    return p1+p2-p3;
```

```
    ...
```

```
}
```

FUNÇÕES

Ex. 1:

```
...  
<a href = "#" onclick = "alo();">Chamar a função</a>  
...
```

alomundo.html

```
function alo() {  
    alert("Link clicado!");  
}
```

alomundo.js

FUNÇÕES

- **Ex. 2:**

```
...  
<form>  
    <input type = "button" value = "Chamar função" onclick =  
    "alo();" />  
</form>  
...
```

alomundo.html

```
function alo() {  
    alert("Link clicado!");  
}
```

alomundo.js

FUNÇÕES

Ex. 3: Passando parâmetros

```
...  
<form>  
  <input type = "button" value = "Chamar função"  
    onclick = "saudacao('jose');"/>  
</form>  
...
```

← saudacao.html

```
function saudacao(nome) {  
  alert("Olá, " + nome);  
}
```

← saudacao.js

FUNÇÕES

Ex. 4: Passando parâmetros de campos de formulário

```
...  
<form>  
  <input type="text" name="txtNome" id = "txtNome"/>  
  <input type="button" name="btn_saudacao"  
    onclick =  
    "saudacao(document.getElementById('txtNome').value);"/>  
</form>
```

```
...  
...  
<form name = "frm">  
  <input type="text" name="txtNome"/>  
  <input type="button" name="btn_saudacao"  
    onclick = "saudacao(frm.txtNome.value);"/>  
</form>  
...
```

FUNÇÕES

Ex. : retornando valores e escrevendo no documento

```
function soma(v1, v2) {  
    return v1 + v2;  
}
```

← soma.js

```
function soma(v1, v2) {  
    document.write(v1 + v2);  
}
```


MODELO DE OBJETOS JAVASCRIPT – OBJETOS

- Ao contrário de uma variável, um objeto pode conter diversos valores e de tipos diferentes armazenados nele (atributos);
- Também possuem funções que operam sobre esses valores (métodos);
- Tanto os atributos, quanto os métodos, são chamados de **propriedades** do objeto.

```
var objeto = new Object();  
// Quando usamos o construtor Object() criamos um objeto  
// genérico
```

MODELO DE OBJETOS JAVASCRIPT - OBJETOS

- Ex.:

```
var nave = {  
    nome: "coração de ouro",  
    propulsao: "Gerador de improbabilidade infinita",  
    dono: "Zaphod Bebbblebrox"  
}  
// Dessa forma, o objeto nave foi criado possuindo os atributos  
// nome, propulsão e dono com seus respectivos valores
```

```
function Carro(modelo, marca, ano, motor) {  
    this.modelo = modelo;  
    this.marca = marca;  
    this.ano = ano;  
    this.motor = motor;  
}  
// Depois para instanciar um objeto, basta usar:  
var car = new Carro("G800" , "Gurgel" , 1976 , 1.0);  
  
// Agora car já possui todos os atributos com dados:  
document.write("Carro: "+car.modelo);  
// o comando acima irá imprimir "Carro: G800"
```

MODELO DE OBJETOS JAVASCRIPT - OBJETOS

- Ex.:

```
var pessoa = {  
  nome: ['Bob', 'Smith'],  
  idade: 32,  
  sexo: 'masculino',  
  interesses: ['música', 'esquiar'],  
  bio: function() {  
    alert(this.nome[0] + ' ' + this.nome[1] + ' tem ' + this.idade + ' anos de  
idade. Ele gosta de ' + this.interesses[0] + ' e ' + this.interesses[1] + '.');  
  },  
  saudacao: function() {  
    alert('Oi! Eu sou ' + this.nome[0] + '.');  
  }  
};
```

MODELO DE OBJETOS JAVASCRIPT - OBJETOS

- Ex.:

```
class Pessoa {  
  
  constructor(nome, idade, sexo, dataDeNascimento) {  
    this.nome = nome;  
    this.idade = idade;  
    this.sexo = sexo;  
    this.dataDeNascimento = dataDeNascimento;  
  }  
  
  apresentar() {  
    console.log(`Ola, meu nome é ${this.nome}, eu tenho ${this.idade} anos`);  
  }  
  
}
```

MODELO DE OBJETOS JAVASCRIPT - OBJETOS

- Ex.:

```
// Uma função fictícia para cálculo de um consumo de combustível
function calc_consumo(distancia) {
    return distancia/(15/this.motor);
}
```

```
// Agora atribuímos a função, sem os argumentos, para a
// propriedade consumo. Considerando o objeto já instanciado
// do exemplo anterior
car.consumo = calc_consumo;
```

```
// Pronto! já podemos invocá-la fazendo:
var gas = car.consumo(200);
// calculando quanto o carro gastaria de
// combustível em 200 quilômetros
```

MODELO DE OBJETOS JAVASCRIPT

- ✓ JavaScript organiza hierarquicamente os objetos em uma página
- ✓ Objetos JavaScript
 - ✓ Objetos de navegação:
 - ✓ window, frame, document, form, button, checkbox, hidden, fileUpload, password, radio, reset, select, submit, text, textarea, link, anchor, applet, image, plugin, area, history, location, navigator.
 - ✓ Objetos JavaScript x tags HTML.
 - ✓ Objetos predefinidos da linguagem
 - ✓ nunca são visíveis na página
 - ✓ por exemplo: String e Array

MODELO DE OBJETOS JAVASCRIPT

✓ Objetos de navegação

✓ criando uma janela

✓ `window.open(URL, Nome, "height=400,width=600")`

MODELO DE OBJETOS JAVASCRIPT

✓ Objeto Document

- ✓ `<BODY> ... </BODY>`

- ✓ propriedades

- ✓ métodos

✓ Objeto Form

- ✓ reflete um formulário HTML em JavaScript

- ✓ `<form> ... </form>`

- ✓ Botão

- ✓ Multiseleção

MODELO DE OBJETOS JAVASCRIPT

- **Objetos predefinidos**
 - **String**
 - ex. length, substring, fontsize, link(URL), toUpperCase
 - **Array**
 - new Array
 - **Date**
 - new Date , getMonth, getCalendarMonth
 - **Math**
 - random, sin, sqrt
 - **Number**
 - new Number(MAX_VALUE)

EVENTOS

- São reações a ações do usuário ou da própria página
ou
- São ocorrências ou acontecimentos dentro de uma página. Ex:
 - Carregar uma página;
 - Clicar em um botão;
 - Modificar o texto de uma caixa de texto;
 - Sair de um campo texto;
 - etc;

EVENTOS

- **onclick:** ocorre quando o usuário clica sobre algum elemento da página

```
...  
<a href = "#" onclick = "alo();">Chamar a função</a>  
...
```

- **onload e onunload:** ocorrem respectivamente quando o objeto que as possuem são carregados (criados) e descarregados

```
...  
<body onload = "bemvindo();" onunload = "adeus();">  
...
```

EVENTOS

```
function bemvindo() {  
    alert("Seja bem vindo.");  
}  
  
function adeus() {  
    alert("Obrigado pela visita.");  
}
```

EVENTOS

- **onmouseover:** é acionado quando o mouse se localiza na área de um elemento
- **onmouseout:** ocorre quando o mouse sai da área de um elemento

```
...  
<a href = "#" onmouseover = "mouseSobre();"   
                                onmouseout = "mouseFora();">  
    Passe o mouse  
</a>  
  
<div id = "resultado"> </div>  
  
...
```

EVENTOS

```
function mouseSobre() {  
    var divResultado = document.getElementById("resultado");  
    divResultado.innerHTML = divResultado.innerHTML +  
        "mouse sobre.<br/>";  
}  
  
function mouseFora() {  
    var divResultado = document.getElementById("resultado");  
    divResultado.innerHTML = divResultado.innerHTML +  
        "mouse fora.<br/>";  
}
```

EVENTOS

- **onsubmit:** usado para chamar a validação de um formulário (ao enviar os dados)
- Para validar um formulário, chamamos uma função por nós definida:
 - Ao chamar a função, usamos a palavra reservada return
- A função, por sua vez, deve retornar true ou false, representando se os dados devem ou não serem enviados. Ex:

```
<form name="frmBusca" action="http://www.google.com/search" method="get" onsubmit =  
"return validaCampo()">
```

```
    Termo: <input type="text" name="q" id = "q" />
```

```
    <input type="submit" name="btnBuscar" value="Buscar"/>
```

```
</form>
```

EVENTOS

```
function validaCampo() {  
    var valor = document.getElementById("q").value;  
  
    if ((valor == null) || (valor == "")) {  
        alert("Preencha o campo de busca");  
        return false;  
    }  
    return true;  
}
```


EVENTOS

- **onfocus:** ocorre quando um controle recebe o foco através do mouse ou do teclado
- **onblur:** ocorre quando um controle perde o foco

...

```
<input type="text" name="txt1" id = "txt1" onfocus = "trataEntrada('txt1')" onblur  
= "trataSaida('txt1')"/>
```

```
<input type="text" name="txt2" id = "txt2" onfocus = "trataEntrada('txt2')" onblur  
="trataSaida('txt2')"/>
```

...

EVENTOS

```
function trataEntrada(id) {  
    var div = document.getElementById("resultado");  
    div.innerHTML = div.innerHTML + id + " ganhou o foco.<br/>";  
}
```

```
function trataSaida(id) {  
    var div = document.getElementById("resultado");  
    div.innerHTML = div.innerHTML + id + " perdeu o foco.<br/>";  
}
```

EVENTOS

- **onkeydown** e **onkeypress**: são semelhantes e ocorrem quando uma tecla é pressionada pelo usuário em seu teclado.
- **onkeyup**: é executado quando a tecla é liberada, ou seja, ela foi pressionada e em seguida liberada.

...

```
<input type="text" name="txtOrigem" id = "txtOrigem"  
onkeydown="copiaTexto('txtOrigem','txtDestino')"/>  
<input type="text" name="txtDestino" id = "txtDestino" />
```

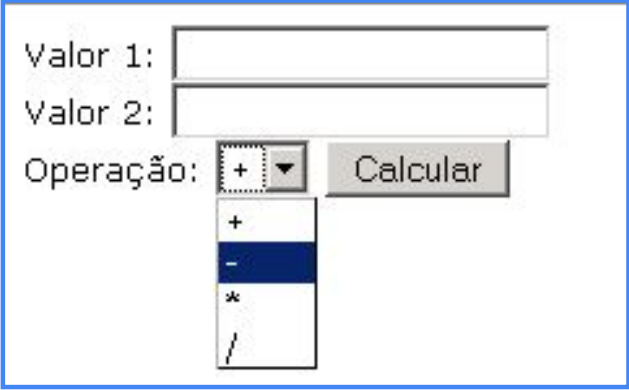
...

EVENTOS

```
function copiaTexto(idOrigem,idDestino) {  
    var txtOrigem = document.getElementById(idOrigem);  
    document.getElementById(idDestino).value = txtOrigem.value;  
}
```

PRÁTICA

- Cria uma página semelhante à figura abaixo e implemente em JS uma calculadora com as 4 operações fundamentais



The image shows a web form for a calculator. It contains two text input fields labeled 'Valor 1:' and 'Valor 2:'. Below these is a label 'Operação:' followed by a dropdown menu. The dropdown menu is currently open, showing four options: '+', '-', '*', and '/'. The '-' option is highlighted with a dark blue background. To the right of the dropdown menu is a button labeled 'Calcular'.

- O valor da caixa select poderá ser obtido da mesma forma que se obtém o valor das caixas de texto
- O resultado do cálculo deve ser exibido com uma função alert
- Use a função parseFloat para converter números reais

PRÁTICA

1. Elabore um formulário HTML que tenha como entrada 3 valores para lados de um triângulo e escreva uma função de nome `tipoTriangulo` que receba 3 parâmetros esses lados de um triângulo e imprima o tipo dele em uma div (equilátero, isósceles ou escaleno).

A passagem dos parâmetros deve ser feita de forma simplificada dentro do HTML no evento onclick de um botão ou link da seguinte forma:

```
<.... onclick = "tipoTriangulo(txtLado1.value, txtLado2.value, txtLado2.value)"...>
```

PRÁTICA

2. Deseja-se calcular a conta de consumo de energia elétrica de uma casa. Para isso, elabore um formulário em HTML que leia a quantidade de Kwh consumidos e o valor unitário do Kwh.

Escreva uma função em JavaScript que faça o cálculo (valor = quantidade x valor unitário) e, caso a quantidade de Kwh ultrapasse 100, o valor do Kwh deve ser acrescido em 25%. Caso ultrapasse 200, o mesmo valor deve ser acrescido em 50%.

Os valores devem ser repassados para uma função em JavaScript conforme o exemplo anterior

DOM – ÁRVORE DE NODOS

- DOM = Representa o documento através de uma árvore de nós (*tree of node*) em que cada objeto possui sua propriedade e método

DOM – ACESSO AOS NODOS

- 3 maneiras

- 1. Usando getElementById()
- 2. Usando getElementsByTagName()
- 3. Navegando pela árvore de nodos usando os relacionamentos entre nodos

```
<html>
  <body>
    <p id="intro">W3Schools example</p>
    <div id="main">
      <p id="main1">The DOM is very useful</p>
      <p id="main2">This example demonstrates
        <b>node relationships</b>
      </p>
    </div>
  </body>
</html>
```

```
var x=document.getElementById("intro");
var text=x.firstChild.nodeValue;
```

DOM – MÉTODOS E PROPRIEDADES

- Propriedades:

- Sendo x um objeto (elemento HTML)

- x.innerHTML – o conteúdo (texto) de x

- x.nodeName – o nome de x

- x.nodeValue – o valor de x

- x.parentNode – o pai de x

- x.childNodes – os filhos de x

- x.attributes – os nodos atributos de x

DOM – MÉTODOS E PROPRIEDADES

- Métodos:

- Sendo x um objeto (elemento HTML)

- `x.getElementById(id)` – obtém o elemento especificado pela ID
 - `x.getElementsByTagName(name)` – obtém todos os elementos especificados pelo nome
 - `x.appendChild(node)` – insere um elemento filho no node x
 - `x.removeChild(node)` – remove um elemento filho do node x

DOM – MÉTODOS

- getElementById

```
<script type="text/javascript">
  <!--
  function showNameJS() {
    var name = document.frm.nomeContato.value;
    alert(name);
  }

  function showNameDOM() {
    var name = document.getElementById('nomeContato').value;
    alert(name);
  }
  //-->
</script>
```

DOM – MÉTODOS

- style

- Altera o estilo de um objeto
- Exemplo de função usada para alterar a propriedade display de um objeto ocultando-o

```
function showHide(id) {  
    var obj = document.getElementById(id);  
    if(obj.style.display == "") {  
        obj.style.display = "none";  
    } else {  
        obj.style.display = "";  
    }  
}
```

DOM – PROPRIEDADES

- Informações dos nodos
- Três propriedades dos nodos importantes:
 - nodeName
 - nodeValue
 - nodeType

VALIDAÇÕES DE FORMULÁRIOS

- Os dados de um formulário devem ser enviados para um servidor.
- Pode-se suavizar o trabalho de um servidor efetuando-se algumas validações no próprio cliente (navegador) com JavaScript
 - Nota:
 - É importante também haver a validação no servidor.
 - A validação com JavaScript serve apenas para amenizar o tráfego de rede com validações simples como campos não preenchidos, caixas não marcadas e etc.

VALIDAÇÕES DE FORMULÁRIOS

- Algumas dicas:
 - Ao se validar um campo, procure sempre obtê-los pelo atributo id
 - Quase todos os elementos do formulário possuem sempre um atributo value, que pode ser acessado como uma String
 - Para verificar um caractere em especial dentro de um valor, use [], pois as Strings são arrays de caracteres
 - As Strings também possuem um atributo length que assim como os arrays, representam o tamanho

VALIDAÇÕES DE FORMULÁRIOS

- **Alguns exemplos de validação:**

- Campos de texto não preenchidos
- Campo de texto com tamanho mínimo e máximo
- Validação de campo de e-mail
- Campos com apenas números em seu conteúdo
- Seleção obrigatória de radio buttons, checkboxes e caixas de seleção

VALIDAÇÕES DE FORMULÁRIOS

- Validação de campo de texto com preenchimento obrigatório:
 - Deve-se validar se:
 - O valor é nulo
 - O valor é uma String vazia
 - O valor é formado apenas por espaço
 - A validação feita para um campo do tipo **text** serve também para um **textarea** e para um **password**
 - Validar espaços pode ser feito usando expressões regulares

VALIDAÇÕES DE FORMULÁRIOS

- Validação de campo de texto com preenchimento obrigatório:

```
function validaCampoTexto(id) {  
    var valor = document.getElementById(id).value;  
    //testa se o valor é nulo, vazio ou formado por apenas espaços em branco  
    if ( (valor == null) || (valor == "") || (/^\s+$/.test(valor)) ) {  
        return false;  
    }  
    return true;  
}
```

VALIDAÇÕES DE FORMULÁRIOS

- Validação de tamanho em campos de texto:
 - É importante validar primeiramente se o campo tem algo preenchido (validação anterior);
 - Pode-se limitar o campo a um tamanho mínimo ou máximo;
 - Usa-se o atributo **length** para se checar o tamanho do campo valor do componente do formulário.

VALIDAÇÕES DE FORMULÁRIOS

- Validação de tamanho em campos de texto:

```
function validaCampoTextoTamanho(id, minimo, maximo) {  
    var valor = document.getElementById(id).value;  
    if (!validaCampoTexto(id)) {  
        return false;  
    }  
  
    if ( (valor.length < minimo) || (valor.length > maximo)) {  
        return false;  
    }  
    return true;  
}
```

VALIDAÇÕES DE FORMULÁRIOS

- Validar para que um campo tenha apenas números:
 - Pode-se validar um campo que deva ser numérico usando-se a função **isNaN** que retorna verdadeiro se um parâmetro não é um número
 - Também é aconselhável validar se o campo contém algum valor

VALIDAÇÕES DE FORMULÁRIOS

- Validar para que um campo tenha apenas números:

```
function validaCampoNumerico(id) {  
    var valor = document.getElementById(id).value;  
    if (isNaN(valor) ) {  
        return false;  
    }  
    return true;  
}
```

VALIDAÇÕES DE FORMULÁRIOS

- Validar se um item foi selecionado numa caixa de seleção ou combo box:
 - Deve-se obter o índice do elemento selecionado através do atributo **selectedIndex**
 - **selectedIndex**: começa do 0 e tem o valor -1 se não houver seleção
 - O índice pode ser nulo se o componente não for do tipo select

VALIDAÇÕES DE FORMULÁRIOS

- Validar se um item foi selecionado numa caixa de seleção ou combo box

```
function validaCampoSelect(id) {  
    var indice = document.getElementById(id).selectedIndex;  
    if ( (indice == null) || (indice < 0) ) {  
        return false;  
    }  
    return true;  
}
```

VALIDAÇÕES DE FORMULÁRIOS

- Validar se uma caixa de checagem (checkbox) está marcada:
 - Deve-se consultar o atributo checked do componente

```
function validaCampoCheckbox(id) {  
    var elemento = document.getElementById(id);  
    if (!elemento.checked) {  
        return false;  
    }  
    return true;  
}
```

VALIDAÇÕES DE FORMULÁRIOS

- Validar se pelo menos um botão de radio de um conjunto foi selecionado:
 - Os campos radio funcionam em conjunto desde que possuam o mesmo atributo name, portanto não se deve consultar pelo id e sim pelo nome pelo método:

`document.getElementsByName(nome);`

- **`getElementsByName(nome)`** retorna um array de elementos com o mesmo nome.
- Esse array deve ser percorrido verificando-se no atributo **checked** se pelo menos um dos botões de radio foram marcados

VALIDAÇÕES DE FORMULÁRIOS

- Validar se pelo menos um botão de radio de um conjunto foi selecionado:

```
function validaCamposRadio(nome) {  
    var opcoes = document.getElementsByName(nome);  
    var selecionado = false;  
    for(var i = 0; i < opcoes.length; i++) {  
        if(opcoes[i].checked) {  
            selecionado = true;  
            break;  
        }  
    }  
    if(!selecionado) {  
        return false;  
    }  
    return true;  
}
```

PRÁTICA

- Nas atividades seguintes:
 - Use uma página HTML e um arquivo de scripts
 - Use o evento **onsubmit** do formulário e uma função de validação que retorne **true** ou **false**
 - Utilize uma página qualquer como **action** do formulário.

PRÁTICA

- Copie o valor de um campo texto para outro caso o campo de origem não esteja vazio. Use o evento onblur do campo de origem
- Valide um campo senha de acordo com seu tamanho:
 - < 3: segurança fraca
 - Entre 3 e 5: segurança média
 - >= 6: segurança forte
- Valide se dois campos do tipo **password** são idênticos
- Valide 3 campos texto que representem dia, mês e ano:
 - Dia: entre 1 e 31
 - Mês: entre 1 e 12
 - Ano: > 1949

O QUE É O AJAX?

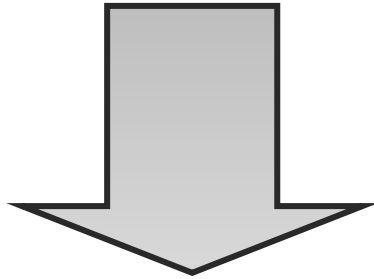
- AJAX é o acrônimo de *Asynchronous javascript and XML*
- Podemos olhar para esta técnica de duas formas distintas:
 - Como sendo um conjunto de tecnologias e padrões
 - Como sendo uma arquitetura

AJAX - AS TECNOLOGIAS

- O AJAX não é por si só uma tecnologia, mas sim um conjunto de tecnologias combinadas(padrões):
 - Utiliza **HTML** e **CSS** para a apresentação de conteúdo
 - Utiliza o **DOM** para oferecer páginas interativas e dinâmicas
 - Utiliza **XML** (entre outros) para troca de dados
 - As trocas assíncronas de dados são efetuadas utilizando o objeto **XMLHttpRequest**
 - Utiliza o **JavaScript** como linguagem, que “combina” todas estas tecnologias

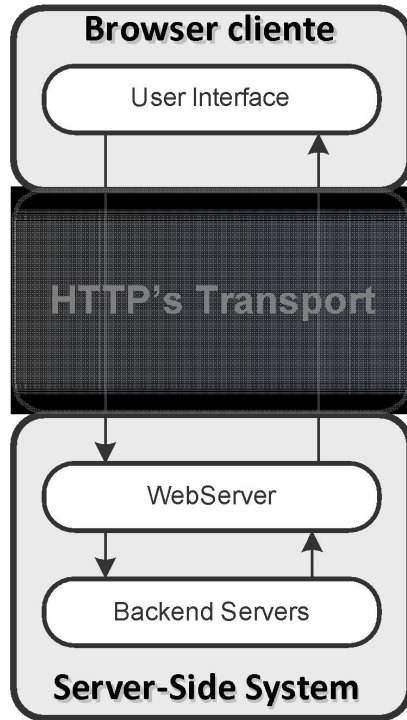
AJAX – A MUDANÇA DE ARQUITETURA

- O AJAX permitiu atualizar a arquitetura base das aplicações WEB
 - *Server Side Events*: Os componentes podem fazer pedidos ao servidor para obter informações, sem forçar o carregamento total da página
 - *Assincronismo*: Os pedidos (parciais) ao servidor não bloqueiam a interação com o navegador.

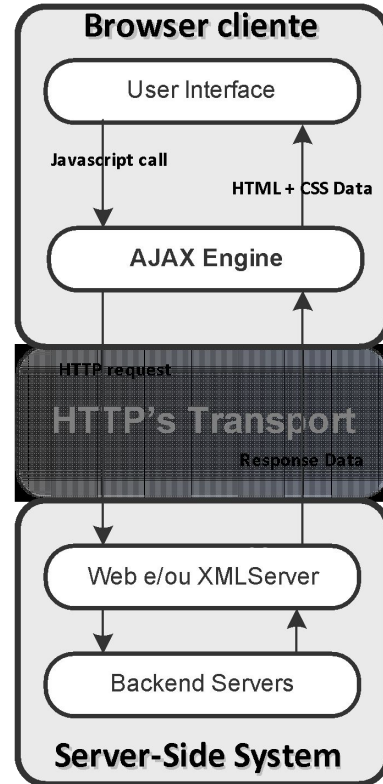


· · · · ·
· **Maior aproximação entre aplicações WEB e Desktop** ·
· · · · ·

MODELO CLÁSSICO vs AJAX



Clássico



AJAX

VANTAGENS

- Aumento da usabilidade da interface das aplicações WEB;
- Possível ter aplicações mais ricas, com mais interações, sem recorrer a *plugins* de terceiros (e.g. Macromedia Flash);
- Aplicações requerem menos largura de banda – apenas é descarregado(baixado) o necessário;
- Interfaces respondem mais rapidamente (embora estejam mais dependentes da velocidade da rede).

EXEMPLO PRÁTICO

- Exemplo de chamada Ajax: AloMundo

```
1 // Firefox, Opera, Chrome, etc...
2 var xhr = new XMLHttpRequest();
3
4
5 ▼ function getAloMundo() {
6     xhr.onreadystatechange = processaDados;
7     xhr.open("GET", "/alomundo.html");
8     xhr.send();
9 }
10
11 ▼ function processaDados() {
12     if (xhr.status == 200 && xhr.readyState == 4)
13         alert(xhr.responseText);
14 }
```

EXEMPLO PRÁTICO

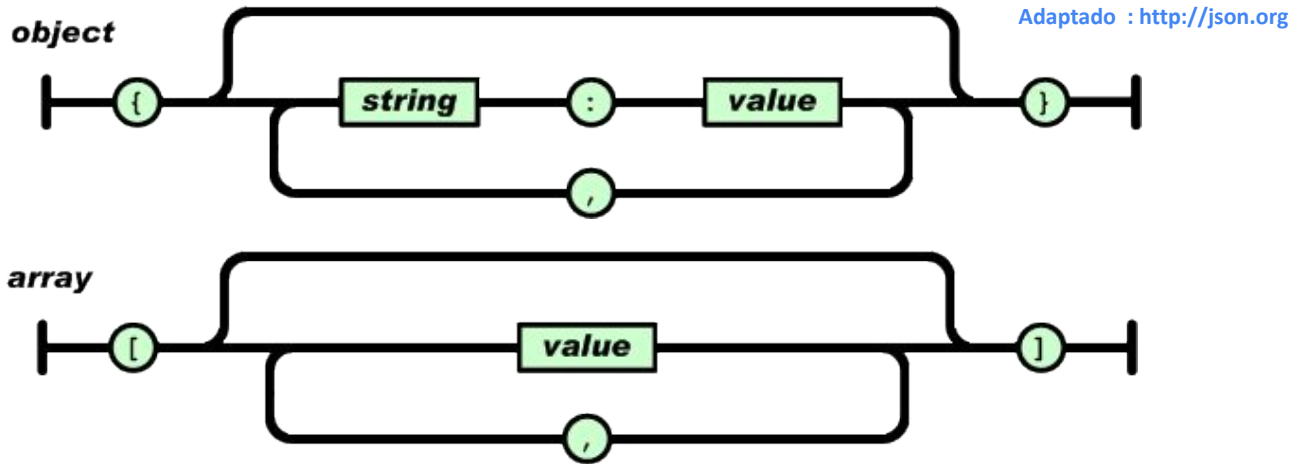
- Exemplo de chamada Ajax: AloMundo

```
1 // Firefox, Opera, Chrome, etc...
2 var xhr = new XMLHttpRequest();
3
4
5 ▼ function getAloMundo() {
6     xhr.onreadystatechange = processaDados;
7     xhr.open("GET", "/alomundo.txt");
8     xhr.send();
9 }
10
11 ▼ function processaDados() {
12     if (xhr.status == 200 && xhr.readyState == 4)
13         alert(xhr.responseText);
14 }
```

JSON (JAVASCRIPT OBJECT NOTATION)

- É um formato baseado na notação literal para objetos do Javascript
 - Standard ECMA-262 3rd Edition - December 1999
- JSON é um formato de texto independente da linguagem
- É utilizado para troca de dados
- JSON contém duas estruturas base:
 - Objeto: Uma coleção de pares nome/valor (tabela de hash)
 - Array: Uma lista ordenada de valores

JSON



JSON Example

```
{  
  "employees": [  
    {"firstName": "John", "lastName": "Doe"},  
    {"firstName": "Anna", "lastName": "Smith"},  
    {"firstName": "Peter", "lastName": "Jones"}  
  ]  
}
```

JSON EM JAVASCRIPT

- Como o JSON se inspirou na notação literal para objetos do ECMAScript, tudo o que é necessário fazer é avaliar a string recebida pelo objeto XMLHttpRequest:

```
var jsonObj = eval( "(" + xhr.responseText + ")" );
```

- Por questões de segurança, em vez de se utilizar a função `eval`, deve-se utilizar um parser JSON, o qual só aceitará texto nesse formato
- Para mais informações: <http://www.json.org/>

EXEMPLO PRÁTICO

- Exemplo de chamada Ajax: wikipediaSearch(JSON)

```
1  // Firefox, Opera, Chrome, etc...
2  var xhr = new XMLHttpRequest();
3
4
5  ▼ function wikipediaSearch() {
6      xhr.onreadystatechange = processaDados;
7      xhr.open("GET",
8          "http://api.geonames.org/wikipediaSearchJSON?
9          q=matematica&maxRows=10&username=demo");
10
11     ▼ function processaDados() {
12         ▼ if (xhr.status == 200 && xhr.readyState == 4) {
13             document.getElementById("txtResultado").innerHTML
14             = xhr.responseText;
15         }
16     }
17 }
```

113

EXEMPLO PRÁTICO

- Exemplo de chamada Ajax: wikipediaSearch(XML)

```
1 // Firefox, Opera, Chrome, etc...
2 var xhr = new XMLHttpRequest();
3
4
5 ▼ function wikipediaSearch() {
6     xhr.onreadystatechange = processaDados;
7     xhr.open("GET",
8         "http://api.geonames.org/wikipediaSearch?
9         q=matematica&maxRows=10&username=demo");
10     xhr.send();
11 }
12
13 ▼ function processaDados() {
14     if (xhr.status == 200 && xhr.readyState == 4) {
15         document.getElementById("txtResultado").innerHTML
16         = xhr.responseText;
17     }
18 }
```

O que é TypeScript?

- Superset do JavaScript desenvolvido pela Microsoft
- Adiciona tipagem estática opcional
- Compila para JavaScript puro
- Ajuda a detectar erros em tempo de desenvolvimento

O que é TypeScript?

- Todo código JavaScript válido também é um código TypeScript válido;
- O TypeScript amplia o JavaScript com tipagem estática, interfaces, enums, generics, entre outros recursos que ajudam na organização e segurança do código.
- Ao final, o código TypeScript é compilado/transpilado para JavaScript puro, para que possa rodar em navegadores e ambientes como o Node.js.

JavaScript vs. TypeScript

- Tipagem: JS – Dinâmica / TS – Estática
- Compilação: JS – Interpretado / TS – Compilado
- IDE: JS – Limitado / TS – IntelliSense
- POO: JS – Limitado / TS – Completo
- Popularidade: JS – Alta / TS – Crescendo

Vantagens do TypeScript

- **Detecção precoce de erros**
- **Autocompletar inteligente**
- **Refatoração mais segura**
- **Melhor documentação do código**
- **Suporte à orientação a objetos**
- **Escalabilidade em grandes projetos**

Tipos Primitivos em TypeScript

- `let nome: string = "Maria";`
- `let idade: number = 25;`
- `let ativo: boolean = true;`
- Tipos: `string`, `number`, `boolean`, `null`, `undefined`, `any`

Interfaces e Tipos Customizados

```
interface Usuario {  
    nome: string;  
    email: string;  
    ativo: boolean;  
}
```


Classes e Orientação a Objetos

```
class Pessoa {  
  constructor(public nome: string) {}  
  cumprimentar() { console.log(`Olá, ${this.nome}`); }  
}
```

Tipos Avançados

- Union types:
 - `string | number`
- Enums:
 - `enum Status { ATIVO, INATIVO }`
- Generics:
 - `function identidade<T>(valor: T): T`

Typescript: Conclusão

- TypeScript melhora qualidade e segurança
- Ideal para projetos de médio e grande porte
- Pode ser adotado de forma gradual

Integração com Projetos Web

- Funciona com React, Angular, Node.js
- Suporte por frameworks modernos
- Compatível com bibliotecas JS via @types

Como começar com TypeScript

- O TypeScript em si não é um runtime — ele é um compilador/transpilador que converte código .ts em JavaScript puro, que depois precisa ser executado por algum ambiente que suporte JS;
- Ferramentas para compilar/transpilar o typescript: [Node.js](#), Bun e Done;
- Usando com o [node.js](#)
 - Instalar: `npm install -g typescript`
 - Criar arquivo .ts e compilar com: `tsc app.ts`
 - Configurar com `tsconfig.json`